

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA0612	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	10% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	CH.Bhavya Sri	Reg. No.:	192411249
Section / Batch:	Slot C	Date:	22.08.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

Question Type	Focus Area	Questions
Comparative Analysis	Comparative analysis of Dynamic Programming and Greedy algorithms for optimization problems.	Q1

SECTION A – Comparative Analysis

Given access frequencies of keys [10, 20, 30, 40], construct an Optimal Binary Search Tree using Dynamic Programming and a Huffman Tree using Greedy approach. Compare tree structures and total search cost. Analyze why Huffman minimizes weighted path length but does not satisfy BST constraints. Evaluate computational complexity of both methods. Discuss applicability in search vs compression problems. Generate insights on choosing DP vs greedy for tree optimization.

Code of Conduct

I CH.Bhavya Sri/192411249____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: CH.Bhavya Sri/192411249____

RUBRICS FOR CALCULATION

Comparative Analysis					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Selection of Studies/Parameters	20%				
Comparison & Differentiation	25%				
Critical Evaluation	25%				
Synthesis & Insight Generation	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

QUESTIONS

Q1

Optimal Binary Search Tree vs.
Huffman Coding (DP vs Greedy
Comparison)

100 marks

0/1 answered

Q1 Optimal Bihary Search Tree vs Huffman Coding (DP vs Greedy Comparison)

100 marks

Given access frequencies of keys [10, 20, 30, 40], construct an Optimal Binary Search Tree using Dynamic Programming and a Huffman Tree using Greedy approach. Compare tree structures and total search cost. Analyze why Huffman minimizes weighted path length but does not satisfy BST constraints. Evaluate computational complexity of both methods. Discuss applicability in search vs compression problems. Generate insights on choosing DP vs greedy for tree optimization.

Your Code

```
1 import heapq
2
3 # Given data from the question
4 keys = [10, 20, 30, 40]
5 freq = [10, 20, 30, 40]
6
7
8 #
9 # OPTIMAL BINARY SEARCH TREE - DYNAMIC PROGRAMMING
10 #
11
12 def optimal_bst(keys, freq):
13     n = len(keys)
14
15     cost = [[0] * n for _ in range(n)]
```

Input

stdin input

Output

Keys: [10, 20, 30, 40]
Access Frequencies: [10, 20, 30, 40]

EXPLANATION

This problem compares two different tree optimization techniques: **Optimal Binary Search Tree (OBST)** using **Dynamic Programming** and **Huffman Coding** using a **Greedy approach**.

In the Optimal Binary Search Tree, the keys are ordered as **10, 20, 30, 40**, and their access frequencies are also given as **10, 20, 30, and 40**. The main objective is to arrange the keys in a Binary Search Tree so that the **total search cost is minimized**. Since a BST must maintain the ordering property, smaller keys are placed on the left and larger keys on the right.

Dynamic Programming is used because different possible roots and subtrees need to be evaluated. The algorithm calculates the minimum cost for different ranges of keys and selects the arrangement that produces the lowest total search cost. This makes OBST suitable for **search-based applications**, such as dictionaries, databases, and indexing systems.

Huffman Coding, on the other hand, uses a **Greedy approach**. It repeatedly combines the two nodes having the smallest frequencies. Its objective is to minimize the **weighted path length**, which results in shorter codes for frequently occurring symbols. Huffman coding is mainly used in **data compression**.

Although Huffman minimizes the weighted path length efficiently, it does **not need to preserve the sorted order of keys**. Therefore, its tree structure may not satisfy the Binary Search Tree property. This is why a Huffman tree cannot generally be directly used as an optimal search tree.

The computational complexity also differs. A standard Dynamic Programming solution for an Optimal BST takes approximately $O(n^3)$ time, while Huffman coding using a priority queue takes $O(n \log n)$ time. Therefore, Huffman is generally faster, but it solves a different optimization problem.

Given access frequencies of keys [10, 20, 30, 40], construct an Optimal Binary Search Tree using Dynamic Programming and a Huffman Tree using Greedy approach. Compare tree structures and total search cost. Analyze why Huffman minimizes weighted path length but does not satisfy BST constraints. Evaluate computational complexity of both methods. Discuss applicability in search vs compression problems. Generate insights on choosing DP vs greedy for tree optimization.

The two approaches also differ in computational complexity. A standard Dynamic Programming solution for the Optimal BST has a time complexity of approximately $O(n^3)$, because it considers different ranges and possible roots. Its space complexity is generally $O(n^2)$ for storing the cost table. Huffman coding using a priority queue has a time complexity of approximately $O(n \log n)$ and requires $O(n)$ additional space.

The applications of these algorithms are different. Optimal BST is useful in **search systems, databases, dictionaries, symbol tables, and indexing**, where maintaining the ordering of keys is important. Huffman coding is mainly used in **data compression, file compression, and communication systems**, where reducing the number of bits required to represent frequently occurring symbols is the main objective.

Therefore, the choice between Dynamic Programming and Greedy algorithms depends on the problem requirements. **Dynamic Programming is suitable when multiple possible arrangements must be compared under structural constraints**, while **Greedy algorithms are suitable when making the best local choice leads to a globally optimal solution**. This comparison demonstrates that the fastest algorithm is not always the best choice; the algorithm must satisfy the specific constraints and objective of the application.

In conclusion, **Dynamic Programming is preferred when ordering and overlapping subproblems must be considered**, as in Optimal BST construction. **Greedy algorithms are preferred when a locally optimal choice leads to a globally optimal solution**, as in Huffman coding. Thus, the choice between DP and Greedy depends on the constraints and objective of the problem.